

Volume 06 - Execution OS Specifications

Version v28 draft

spec-06-exec-01-execution-os-specification.md

Execution OS Specification

Title: Execution OS Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-01

Related Blueprint Documents

- MAL-V03-B03-09-001
- MAL-V03-B03-08-001
- MAL-V03-B03-20-001
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the orchestration layer that turns enterprise missions into governed task graphs, scheduled work, coordinated swarms, and observable execution outcomes. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for execution os inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the execution os capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.
- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.

- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- `execution.mission.accepted`
- `execution.task.ready`
- `execution.task.dispatched`
- `execution.task.completed`
- `execution.task.failed`
- `execution.mission.stalled`
- `execution.recovery.started`
- `execution.recovery.completed`
- `execution.acceptance.passed`

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.

- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-01	Defines v28 Execution OS requirements for Execution OS.
MAL-V03-B03-09-001	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Execution OS.

Chief Orchestration Specification

Title: Chief Orchestration Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-02

Related Blueprint Documents

- MAL-V03-B03-03-002
- MAL-V03-B03-18-001
- MAL-V03-B03-20-001
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines how the enterprise Chief converts strategic intent into executable missions, authority boundaries, priority rules, and governance checkpoints. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for chief orchestration inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the chief orchestration capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.

- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.
- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.

- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- `execution.mission.accepted`
- `execution.task.ready`
- `execution.task.dispatched`
- `execution.task.completed`
- `execution.task.failed`
- `execution.mission.stalled`
- `execution.recovery.started`
- `execution.recovery.completed`
- `execution.acceptance.passed`

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.

- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.

- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-02	Defines v28 Execution OS requirements for Chief Orchestration.
MAL-V03-B03-03-002	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Chief Orchestration.

Mission Manager Specification

Title: Mission Manager Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-03

Related Blueprint Documents

- MAL-V03-B03-03-003
- MAL-V03-B03-08-002
- MAL-V06-RUNTIME-SPEC-010
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the mission control component responsible for mission intake, decomposition, state tracking, escalation, and completion evidence. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for mission manager inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the mission manager capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-03	Defines v28 Execution OS requirements for Mission Manager.
MAL-V03-B03-03-003	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Mission Manager.

Swarm Engine Specification

Title: Swarm Engine Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-04

Related Blueprint Documents

- MAL-V03-B03-09-001
- MAL-V03-B03-03-006
- MAL-V03-B03-03-011
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the runtime responsible for forming, directing, monitoring, and dissolving multi-agent swarms for complex execution. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for swarm engine inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the swarm engine capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-04	Defines v28 Execution OS requirements for Swarm Engine.
MAL-V03-B03-09-001	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Swarm Engine.

Team Coordination Specification

Title: Team Coordination Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-05

Related Blueprint Documents

- MAL-V03-B03-03-004
- MAL-V03-B03-03-005
- MAL-V03-B03-02-011
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines coordination rules for department managers, team leaders, agents, reviewers, validators, and supporting services during execution. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for team coordination inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the team coordination capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-05	Defines v28 Execution OS requirements for Team Coordination.
MAL-V03-B03-03-004	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Team Coordination.

Agent Routing Specification

Title: Agent Routing Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-06

Related Blueprint Documents

- MAL-V03-B03-03-007
- MAL-V03-B03-05-004
- MAL-V03-B03-05-006
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines assignment, routing, and rerouting of tasks to agents based on capability, availability, policy, context, and performance. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for agent routing inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the agent routing capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-06	Defines v28 Execution OS requirements for Agent Routing.
MAL-V03-B03-03-007	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Agent Routing.

Task Graph Specification

Title: Task Graph Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-07

Related Blueprint Documents

- MAL-V03-B03-08-003
- MAL-V03-B03-09-004
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the directed execution graph used to represent tasks, dependencies, gates, retries, evidence, and completion state. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for task graph inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the task graph capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-07	Defines v28 Execution OS requirements for Task Graph.
MAL-V03-B03-08-003	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Task Graph.

Mission Graph Specification

Title: Mission Graph Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-08

Related Blueprint Documents

- MAL-V03-B03-08-001
- MAL-V03-B03-15-001
- MAL-V03-B03-20-001
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the higher-level graph connecting objectives, milestones, task graphs, swarms, constraints, and acceptance evidence. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for mission graph inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the mission graph capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-08	Defines v28 Execution OS requirements for Mission Graph.
MAL-V03-B03-08-001	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Mission Graph.

Distributed Scheduler Specification

Title: Distributed Scheduler Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-09

Related Blueprint Documents

- MAL-V03-B03-07-011
- MAL-V03-B03-19-001
- MAL-V06-RUNTIME-SPEC-011
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines scheduling rules for concurrent, delayed, dependent, prioritized, and resource-constrained work across runtimes. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for distributed scheduler inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the distributed scheduler capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-09	Defines v28 Execution OS requirements for Distributed Scheduler.
MAL-V03-B03-07-011	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Distributed Scheduler.

Parallel Executor Specification

Title: Parallel Executor Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-10

Related Blueprint Documents

- MAL-V03-B03-09-002
- MAL-V03-B03-07-007
- MAL-V06-RUNTIME-SPEC-014
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines controlled parallel execution of independent tasks with bounded concurrency, isolation, aggregation, and recovery. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for parallel executor inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the parallel executor capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-10	Defines v28 Execution OS requirements for Parallel Executor.
MAL-V03-B03-09-002	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Parallel Executor.

Long Running Missions Specification

Title: Long Running Missions Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-11

Related Blueprint Documents

- MAL-V03-B03-08-004
- MAL-V03-B03-20-002
- MAL-V06-RUNTIME-SPEC-010
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines durable execution for missions that span multiple sessions, agents, data refreshes, approvals, and recovery cycles. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for long running missions inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the long running missions capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-11	Defines v28 Execution OS requirements for Long Running Missions.
MAL-V03-B03-08-004	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Long Running Missions.

Stall Detection Specification

Title: Stall Detection Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-12

Related Blueprint Documents

- MAL-V03-B03-17-001
- MAL-V03-B03-07-013
- MAL-V05-STD-015
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines detection of blocked, idle, looping, waiting, degraded, and low-progress execution states. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for stall detection inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the stall detection capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-12	Defines v28 Execution OS requirements for Stall Detection.
MAL-V03-B03-17-001	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Stall Detection.

Recovery Engine Specification

Title: Recovery Engine Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-13

Related Blueprint Documents

- MAL-V03-B03-07-014
- MAL-V03-B03-19-005
- MAL-V06-RUNTIME-SPEC-014
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines recovery plans for failed tasks, failed agents, missing dependencies, expired context, and partial mission completion. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for recovery engine inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the recovery engine capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-13	Defines v28 Execution OS requirements for Recovery Engine.
MAL-V03-B03-07-014	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Recovery Engine.

Self Healing Specification

Title: Self Healing Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-14

Related Blueprint Documents

- MAL-V03-B03-20-003
- MAL-V03-B03-21-001
- MAL-V05-STD-014
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines automated repair behaviors that restore execution health without violating governance, security, or acceptance constraints. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for self healing inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the self healing capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-14	Defines v28 Execution OS requirements for Self Healing.
MAL-V03-B03-20-003	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Self Healing.

Execution Testing Specification

Title: Execution Testing Specification

Version: v28 draft

Status: Draft

Specification ID: MAL-V06-EXEC-SPEC-15

Related Blueprint Documents

- MAL-V03-B03-09-019
- MAL-V05-STD-013
- MAL-V06-RUNTIME-SPEC-015
- MAL-V06-RUNTIME-SPEC-001
- MAL-V06-RUNTIME-SPEC-008
- MAL-V06-RUNTIME-SPEC-013

Purpose

Defines the required verification suite for mission orchestration, swarm execution, scheduling, recovery, and self-healing. This specification makes the behavior executable by defining required responsibilities, interfaces, events, persistence rules, failure handling, and acceptance evidence for Mariam AI Enterprise OS.

Scope

- Covers runtime behavior required for execution testing inside the enterprise execution layer.
- Includes mission orchestration, task graph coordination, swarm participation, scheduling signals, execution state, recovery behavior, and testing obligations where applicable.
- Excludes UI presentation details, vendor-specific model prompts, and implementation code that belongs in product repositories.

Responsibilities

- Maintain authoritative execution state for the execution testing capability.
- Enforce mission policy, role authority, capability matching, dependency order, and acceptance gates before work is marked complete.
- Coordinate with Runtime Ecosystem, AI Society, Workflow System, Knowledge System, Capability System, and Enterprise Governance documents.
- Emit durable events for lifecycle transitions, scheduling decisions, execution attempts, failures, recoveries, and acceptance outcomes.
- Preserve evidence that allows reviewers to reconstruct why a mission, task, route, or recovery action happened.

Functional Requirements

- The component SHALL accept mission, task, swarm, or schedule requests only through validated public interfaces.
- The component SHALL create deterministic identifiers for missions, task graph nodes, execution attempts, agent assignments, and recovery plans.

- The component SHALL validate dependencies before dispatching work and SHALL prevent execution of blocked graph nodes.
- The component SHALL support pause, resume, cancel, retry, reassign, escalate, and complete operations with explicit state transitions.
- The component SHALL record decision context, selected agents or runtimes, dependency status, and acceptance evidence for every meaningful transition.
- The component SHALL expose query operations for current state, historical attempts, blocked items, active schedules, and recovery status.
- The component SHALL integrate with governance checkpoints when mission risk, cost, security level, or confidence thresholds require approval.

Non-Functional Requirements

- State transitions MUST be idempotent and safe under duplicate event delivery.
- Scheduler and executor decisions MUST be explainable from persisted inputs, policies, capability scores, and dependency state.
- Execution state MUST tolerate process restarts without losing active mission context.
- The implementation SHOULD support bounded parallelism and backpressure so one mission cannot starve the enterprise queue.
- The implementation MUST keep sensitive mission data isolated by tenant, department, project, and agent authorization scope.

Inputs

- Mission intents, objectives, constraints, acceptance criteria, budgets, deadlines, and priority signals.
- Task graph definitions, dependency edges, required capabilities, role constraints, and runtime requirements.
- Agent availability, capability scores, workload, health, memory scope, and governance permissions.
- Runtime events from tools, plugins, connectors, models, workflows, and storage systems.
- Reviewer decisions, approval results, escalation responses, and validation outcomes.

Outputs

- Mission records, task graph records, execution plans, schedules, agent assignments, and recovery plans.
- Lifecycle events for mission accepted, task ready, task dispatched, task completed, task failed, mission stalled, mission recovered, and mission accepted.
- Operational metrics covering throughput, latency, queue depth, retry rate, stall rate, recovery success, and acceptance pass rate.
- Evidence bundles containing inputs, outputs, decisions, reviewer notes, validation results, and traceability links.

Public Interfaces

- `submit_mission(intent, constraints, acceptance_criteria)` for creating governed mission execution.
- `get_mission_state(mission_id)` for retrieving mission status, graph progress, active blockers, and evidence.
- `update_task_state(task_id, transition, evidence)` for controlled lifecycle changes.
- `request_recovery(scope_id, reason, policy)` for initiating recovery against failed or stalled execution.
- `list_execution_work(queue_filter)` for operational dashboards and authorized schedulers.

Internal Interfaces

- Runtime Manager interface for dispatching executable work and receiving attempt results.
- Capability Registry interface for matching tasks to agents, tools, plugins, connectors, and providers.
- Event Bus interface for publishing durable execution lifecycle events.
- Knowledge Store interface for retrieving mission context and storing evidence.
- Governance service interface for approval, escalation, risk, and policy decisions.

Events

- execution.mission.accepted
- execution.task.ready
- execution.task.dispatched
- execution.task.completed
- execution.task.failed
- execution.mission.stalled
- execution.recovery.started
- execution.recovery.completed
- execution.acceptance.passed

Storage Requirements

- Store mission records, task graph nodes, dependency edges, assignments, schedules, attempts, logs, evidence, and recovery plans.
- Store event offsets or idempotency keys to prevent duplicate state transitions.
- Store policy versions and capability snapshots used for routing and scheduling decisions.
- Retain execution evidence according to governance, audit, customer, and security retention rules.

Security Requirements

- Enforce tenant, department, project, mission, role, and agent-level authorization on all read and write operations.
- Prevent unauthorized agents, runtimes, tools, plugins, or connectors from receiving restricted mission context.
- Require signed or authenticated events for state-changing execution operations.
- Redact secrets and regulated data from logs, metrics labels, event payloads, and exported evidence.
- Require governance approval for destructive, externally visible, financial, or privileged execution actions.

Observability Requirements

- Emit metrics for mission count, active task count, queue depth, scheduling latency, execution latency, retry rate, stall rate, recovery rate, and acceptance failure rate.
- Emit structured logs with mission ID, task ID, graph ID, agent ID, runtime ID, transition, decision reason, and correlation ID.
- Provide traces across mission submission, planning, routing, scheduling, execution, review, recovery, and acceptance.
- Provide dashboards for active missions, blocked graph nodes, failing agents, overloaded runtimes, and recovery outcomes.

Failure Modes

- Mission graph contains missing dependencies or circular dependencies.
- Assigned agent becomes unavailable, unauthorized, overloaded, or fails validation.
- Runtime, tool, plugin, connector, provider, or storage dependency becomes unavailable.
- Task output fails acceptance criteria or conflicts with mission constraints.
- Execution stalls because of waiting approvals, missing context, repeated retries, or unresolved dependency failures.

Recovery Rules

- Retry only idempotent or explicitly recoverable operations without manual approval.
- Reassign work when capability requirements can be satisfied by another authorized agent or runtime.
- Pause dependent graph branches until failed prerequisites are resolved.
- Escalate to governance when recovery changes mission scope, risk, budget, deadline, or external side effects.
- Preserve failed attempts and recovery evidence instead of overwriting historical execution records.

Test Requirements

- Unit tests **MUST** validate state transition rules, dependency validation, idempotency keys, authorization checks, and event payload schemas.
- Integration tests **MUST** cover mission submission, graph creation, scheduling, execution, failure, recovery, and acceptance.
- Load tests **SHOULD** cover concurrent missions, large task graphs, high event volume, and runtime backpressure.
- Security tests **MUST** verify restricted context isolation, approval gates, unauthorized transitions, and audit evidence.
- Chaos tests **SHOULD** simulate failed agents, failed runtimes, delayed events, duplicate events, and storage interruptions.

Acceptance Criteria

- All required public and internal interfaces are documented and versioned.
- The specification maps to related Blueprint and Runtime Specification documents.
- Mission and task lifecycle states are persisted, queryable, observable, and recoverable.
- Failure and recovery behavior is deterministic enough to test and audit.
- Security, governance, observability, and traceability requirements are enforceable before implementation begins.

Implementation Notes

- Prefer append-only execution history with derived current state for auditability.
- Treat graph transitions as commands validated by policies, not as direct mutable database updates.
- Use bounded worker pools and explicit scheduling leases to avoid duplicate active execution.
- Keep event schemas stable and backward compatible across v28 draft refinements.
- Implement minimal vertical slices first: mission intake, task graph, scheduling, executor result, recovery, and acceptance evidence.

Traceability

Source	Relationship
MAL-V06-EXEC-SPEC-15	Defines v28 Execution OS requirements for Execution Testing.
MAL-V03-B03-09-019	Primary Blueprint source.
MAL-V06-RUNTIME-SPEC-001	Runtime integration source.
MAL-V05-STD-013	Testing standard reference.
MAL-V05-STD-015	Observability standard reference.

Version History

Version	Date	Status	Notes
v28 draft	2026-06-29	Draft	Initial Execution OS specification for Execution Testing.